

工程问答 — 面试题提取 × miniagent 实战回答

题目来自 DeepSeek Agent 岗三面连环追问 (工程细节/系统设计/赛道认知)。共 16 题, 每题标注: **【已有机制】** 现场验证过 **【针对性补全】** 本轮

新增 {kimi/Hermes 参照}

一面·工程细节题

Q1 上下文用的是 message window 还是 token window? 线上为什么这么选?

【已有机制】 token window。 context.py 以估算 token 数 (chars/4) 对 30 万窗口设 85% 触发线,message window 无法表达 “10 条消息里有一条 240KB 的工具输出” 这种真实分布—T5 实测单条 read_file 结果就是 25 万字符。选 token window 的代价是估算漂移, 工业解法是 kimi 的 “实测锚点 + 估算尾部” 双层计量,miniagent 停在单层粗估 + 85% 保守阈值 + 溢出自愈兜底 (Q4)。{kimi: 双层计量·Hermes: 三套信号协调}

Q2 会话持久化的过期策略怎么定? 遇到过并发会话冲突吗?

【针对性补全】 存储选型是 **JSONL 文件而非 Redis**(教程准则四: 只要恢复 → 追加日志够用, 要上检索才换库),过期策略:session.cleanup_old_sessions(word ttl_days=30), 按文件 mtime TTL 清理,CLI --cleanup-days N。

并发冲突遇到过, 分两处解决:□ 多会话同 workdir 写 MEMORY.md → 行级原子追加 + tmp+rename 原子替换截断;□ 多子代理共享上下文压缩 → 物理隔离 (各自独立 transcript), 参考 Hermes #38727 的尸检。{kimi: AppendLogStore tmp+rename cutover ·Hermes: SQLite 压缩锁 TTL 300s}

Q3 长期记忆用向量库吗? 记忆越来越多、检索越来越不准怎么办?

【针对性补全】 不用向量库—miniagent 的记忆是有 8000 字符硬上限的平文件 (容量即精度: 上限强制淘汰旧条目, 检索域永远小到关键词足够准)。本轮补了 memory_search 工具 (关键词检索, 只读)。真正的防线在写入侧:provenance 时间戳溯源 + 注入安检, 毒条目不进检索域。什么时候该上向量库: 记忆条目破千、跨项目复用—教程里 Hermes 用 SQLite FTS5(含 CJK trigram) 也是先 BM25 后向量, 且曾砍掉摘要 LLM 路径回归纯 BM25。{Hermes: FTS5+trigram, “session 霸榜” 教训}

Q4 上下文 token 溢出的兜底方案是什么?

【针对性补全】 三层:□ 预防—每步 preflight 估算超 85% 即压缩;□ **自愈 (本轮新增)**—provider 报 context/overflow/length 错误时 _chat() 捕获,force=True 强制压缩后重试, 连续 3 次放弃 (对齐 kimi 溢出自愈上限);□ 兜底—摘要调用失败 fail-open 保原会话, 预算耗尽还有 grace 收尾。单测 TestOverflowHeal ×2 覆盖成功与放弃两条路径。{kimi: 错误处理器认领 413, 下调有效窗口 ×0.85}

Q5 单用户日均 token 成本多少? 百万级流量怎么控制?

【针对性补全】 可测才能答: 本轮给每步 LLM 调用加了**用量账本** (prompt/completion/calls 累加,turn 结束落 transcript usage 事件)。现场实测: 一次 “建文件 + 运行 + 查记忆” 的轻任务 = **5,838 prompt + 189 completion / 4 次调用**; 一次完整项目任务 (T1,39 测试项目) 约 14 次工具调用、~10 次 LLM 调用。百万级控制三板斧全在架构里:□ 缓存宪法 (前缀只追加,DashScope 命中缓存约 1/10 价);□ 结果治理 (50K 落盘只回 2K,T5 实测 25.6 万字符 →2,130);□ 预算硬墙 (max_steps + 子代理独立预算)。{kimi: 分段缓存 + 末尾注入·Hermes: 4 断点 + 冻结快照}

Q6 调用拦截: 用户恶意刷长文本怎么限制?

【针对性补全】 三层拦截:□ 入口—AGENT815_MAX_INPUT_CHARS(默认 20 万字符) 超限直接拒绝, **不消耗任何 LLM 调用** (单测验证 FakeLLM 零调用);□ 工具侧—bash 120s 超时、输出 30K 截断、大结果 50K 落盘;□ 循环侧—max_steps 硬墙 + stop hook 一次性门锁防 “永不结束”。

Q7 为什么用基座模型不用微调? 拿什么数据做判断?

【已有机制】 选基座 (qwen3.7-max) 因为 Harness 架构把行为约束都放在了**工程层** (system prompt 纪律 + 工具协议 + 权限链), 这些不需要权重级定制; 微调的适用面是输出格式/领域风格稳定且高频的场景,coding agent 的需求是通用推理 + 工具调用可靠性。判断数据:□ 工具调用成功率 (实测 73 次调用 0 协议错误);□ 任务完成率 (8 个现场任务 8/8 exit 0);□ 失败自愈率 (T1/T3 测试失败 →edit_file 修复 → 复测通过);□ 单任务 token 成本 (Q5 账本)。微调唯一可能值回票价的点是把 BASE_PROMPT 的纪律烧进权重省 system prompt token—但会损失迭代速度, 不划算。

Q8 讲一个线上故障排查: 改了哪块逻辑, 效果多少?

【已有机制】 单 user 会话压缩失效。现象: 小窗口压力测试 (MAX_CONTEXT=1500) 下压缩事件恒为 0。排查: 逐步重放真实 transcript 计算各

preflight 时点估算, 确认第 3 条消息起已超阈值却不触发 → 定位到 context.py “最后一条 user 必须在 tail” 守护规则: 一次性任务唯一 user 消息在 head(索引 1), 守护把 cut 强制拉回 1,middle 恒空, 压缩静默放弃。修复: 守护仅在 last_user_idx ≥ 2 生效。效果: 同任务压缩触发从 0 → 2 次, 免疫摘要正确注入; 回归测试 test_single_user_message_still_compacts 防复发。教训: 守护规则必须考虑 “被保护对象已在保护区” 的边界。

二面·系统设计题

Q9 系统模块怎么拆分? 职责与通信? 为什么这么拆?

【已有机制】 11 个模块按 “窄腰核心 + 边上插拔” 拆 (见模块设计页): loop 是唯一有状态核心, 其余模块通过三个窄接口通信 — □ 工具协议 (schema+handler 字典); □ 事件表 (hooks 5 事件); □ 消息列表 (唯一共享状态, 只追加)。拆分依据是教程两条宪法: 核心小到可以穷举测试 (loop 168 行), 换 provider/换存储/换表面不动核心。通信不走消息总线而共享不可变前缀的消息列表, 是为了缓存宪法 — 任何总线式动态注入都会打碎前缀缓存。

Q10 怎么判断单工具简单任务 vs 多步拆解复杂任务? 拆解粒度?

【已有机制】 miniagent 走 kimi 路线: **拆解权交给模型, 工程层只设边界** — 模型自主决定是否连续调工具 (终止由模型决定), 工程层的控制是: max_steps 硬墙 (60)、子代理独立预算 (30)、delegate_task 让模型把 “探索性子任务” 显式派生出去 (粒度信号: 会污染主线上文活的活就派生, 探索日志只回蒸馏报告)。Hermes 路线是预算 refund (模型写代码循环调工具不侵蚀预算) — 两种都验证了 “不要在工程层替模型做拆解判断”, 那会变成永远追不上模型能力的启发式。

Q11 任务执行一半失败: 重试、回滚还是重新规划? 触发条件?

分四层, 每层触发条件显式:

失败类型	策略	触发条件
工具执行失败	错误文本回填, 模型自行重试或换路 (T1 实测: 测试失败 → edit_file → 复测通过)	任何 tool exception
429/5xx/网络超时	指数退避重试 (×2 上限 30s + 20% 抖动 + Retry-After), 最多 5 次	HTTP 状态码/连接异常
上下文溢出	强制压缩后重试, 连续 3 次放弃	错误文本含 context/overflow/length
权限拒绝/hook 阻断	不重试不绕过 , 原因回填让模型换方案 (T4b 实测 pip 被禁 → 标准库完成)	sandbox/hook veto

回滚在最小设计里刻意没有 (文件操作无事务), 对应演化级是 kimi 的 undo/wire Op 重放 — 痛了再加。

Q12 多工具怎么联动? 权限怎么管控? 怎么防越权?

【已有机制】 联动: 工具是纯字典注册表, 联动顺序由模型按 schema 描述编排; 结果治理保证任何单工具输出不灌爆窗口。权限五道防线, 顺序即优先级: □ **hardline 硬线** (rm -rf /、fork bomb、写 authorized_keys, **yolo 也不放行** — 用户不能授权无恢复路径的破坏); □ 去混淆 (NFKC 全角/\$IFS/引号拼接) 防绕过; □ 未知工具 fail-closed; □ 只读/写执行三分级 + ask/auto/yolo 模式; □ workdir 围栏 + 子进程环境变量剥离 (KEY/TOKEN 类, 实测工作区零密钥痕迹)。防越权的价值观: **危险检测先于用户授权**。{Hermes: hardline 先于 yolo bypass · GHSA-rhgp 凭据隔离}

三面·认知题 (回答框架, 以本项目为论据)

Q13 Agent 是长期风口还是短期热点? 长期落地方向?

长期。论据: 模型能力每代跃迁, 但教程里两个工业 codebase 的厚度证明 **工程表现的上下限由 Harness 决定** — 同一模型接不同 Harness, 安全性/成本/长任务存活率差一个数量级, 这层工程不会随模型变强而消失 (缓存宪法、权限纵深、状态可恢复都是模型外问题)。落地方向: 有人值守的编程 CLI (kimi 型) 先跑通, 无人值守的多表面平台 (Hermes 型) 跟随, 自学习闭环 (记忆/技能双闭环) 是拉开差距的下半场。

Q14 和普通大模型开发者比, 核心竞争力?

能从“事故”反推架构。miniagent 每个模块都能回答“防什么事故、对应演化阶梯第几级、kimi/Hermes 怎么解”—demo 开发者堆框架, 工程师管爆炸半径 (hardline 先于 yolo)、管成本 (缓存宪法)、管最坏情况下限 (fail-open 方向都是显式决策)。

Q15 1-2 年内最大瓶颈是模型能力还是工程能力?ToB 还是 ToC 先跑通?

工程能力。模型已经“够用”(实测 qwen3.7-max 零协议错误完成 8 个完整项目), 卡脖子的是: 长会话成本 (压缩与缓存的博弈)、无人值守安全 (纵深防御)、状态可恢复 (断点续跑)。ToB 先跑通—容错预算高、场景边界清晰、权限模型可预先声明;ToC 的越权风险面不可控。

Q16 从 0-1 带一条 Agent 业务线, 第一步做什么?

先定形态再写代码 (教程决策树 Q1): 有人盯着还是无人值守? 这一个问题决定主循环形状 (插件链 vs 预算硬墙)、安全厚度 (规则链 vs 纵深栈)、存储选型 (JSONL vs SQLite+FTS5)。招人按模块招 (循环/上下文/安全/记忆技能/表面), 每人负责一条演化阶梯的爬升路线, 评审标准就是“每份复杂性答得出防什么事故”。

本页为 miniagent 实战报告独立附页 (源:interview.html)。完整版见 miniagent_report_full.pdf(六大板块合并)。